

How to Create a Workload Cluster

A “workload cluster” on vSphere with Tanzu is a real Kubernetes cluster (TKG — Tanzu Kubernetes Grid) provisioned on top of the Supervisor. The Supervisor is the control plane that creates and manages these clusters; the TKG clusters are the ones you actually deploy your apps to.

This doc covers creating one from scratch, assuming the Supervisor in `PREREQUISITES.md` is already up and running.

Prerequisites

Before creating a workload cluster, confirm all five exist:

Requirement	How to verify
Supervisor cluster <code>RUNNING</code>	<code>curl -sk -H "vmware-api-session-id: \$SESSION" https://\$VCENTER/api/vcenter/namespace-management/clusters/<cluster-id></code> → <code>"config_status": "RUNNING"</code>
A vSphere Namespace	<code>kubectl config get-contexts</code> (after <code>kubectl vsphere login</code>) shows at least one user-owned context like <code>test-ns</code>
Storage policy attached to the namespace	The namespace's <code>storage_specs</code> includes the <code>supervisor-storage</code> policy (vSphere UI → Namespaces → <i>namespace</i> → Storage)
Default Kubernetes Content Library	Cluster reports <code>content_libraries: [{kubernetes.vmware.com}]</code> (vSphere UI → Menu → Content Libraries)
<code>kubectl</code> + <code>kubectl vsphere</code> plugin installed	<code>kubectl version --client</code> and <code>kubectl vsphere version</code> both succeed

If any of these is missing, see `PREREQUISITES.md` (Supervisor setup) or `TROUBLESHOOTING.md` (DNS, login, namespace creation).

Step 1 — switch to the namespace context

After `kubectl vsphere login`, you'll have one context per namespace you have access to. Switch to your user namespace:

```
kubectl config use-context test-ns
```

All subsequent `kubectl` commands target that namespace's slice of the Supervisor.

Step 2 — see what's available

```
# OS images available — these are TKG release versions:
kubectl get tanzukubernetesreleases
# Shorter alias: kubectl get tkr
# Example output:
#   NAME                                VERSION                                READY
#   v1.28.8---vmware.1-fips.1-tkg.1     v1.28.8+vmware.1-fips.1-tkg.1       True
#   v1.27.13---vmware.1-fips.1-tkg.1    v1.27.13+vmware.1-fips.1-tkg.1     True

# VM size classes (control plane + workers pick from these):
kubectl get virtualmachineclass
# Common values:
#   best-effort-xsmall    2 CPU, 2 GB
#   best-effort-small    2 CPU, 4 GB
#   best-effort-medium   2 CPU, 8 GB
#   guaranteed-small     2 CPU, 4 GB (reserved)

# Storage classes (storage policies surfaced as Kubernetes StorageClasses):
kubectl get storageclass
# Should include supervisor-storage matching the lab policy.
```

If `kubectl get tkr` returns empty, the content library hasn't finished syncing yet (5-10 min after Supervisor reaches RUNNING) or the namespace isn't bound to the library. See Troubleshooting below.

Step 2.5 — pick your sizing

The cluster CR you'll apply needs:

- `version:` — a real `tkr` from the list above
- `controlPlane.replicas:` — `1` for lab, `3` for HA
- `workers.machineDeployments[].replicas:` — `1` — `N` worker count
- `vmClass:` — applies to both CP and workers (overridable per-pool)

- `storageClass:` — must be one of `kubectl get storageclass`

Lab-friendly minimum: 1 CP + 1 worker × `best-effort-xsmall` → ~6 GB RAM total.

Production-ish: 3 CP × `best-effort-small` + 3 workers × `best-effort-medium` → ~36 GB.

Step 3 — apply the Cluster spec

vSphere 9.0+ uses the ClusterAPI-based `Cluster` resource with a built-in `ClusterClass` named `tanzukubernetescluster`. A ready-to-edit copy lives at `terraform/examples/cluster/tkc.yaml`; the same content shown here for reference:

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata:
  name: tkc-01
  namespace: test-ns
spec:
  clusterNetwork:
    services:
      cidrBlocks: ["10.96.0.0/16"]
    pods:
      cidrBlocks: ["10.244.0.0/16"]
      serviceDomain: cluster.local
  topology:
    class: tanzukubernetescluster
    version: v1.28.8+vmware.1-fips.1-tkg.1 # ← MUST match a `kubectl get tkr`
    controlPlane:
      replicas: 1 # 3 for HA
      metadata: {}
    workers:
      machineDeployments:
        - class: node-pool
          name: node-pool-1
          replicas: 2
          variables:
            overrides:
              - name: vmClass
                value: best-effort-small
              - name: storageClass
                value: supervisor-storage
      variables:
        - name: vmClass
          value: best-effort-small
        - name: storageClass
          value: supervisor-storage
        - name: defaultStorageClass
          value: supervisor-storage
```

Apply:

```
kubectl apply -f tkc.yaml
```

`Cluster` is the only object you create; the ClusterClass handles generating the underlying control-plane and machine-deployment resources.

Understanding the `clusterNetwork` block

Two different controllers hand out IPs from those CIDRs:

Field	Who allocates	How
<code>services.cidrBlocks</code>	kube-apiserver	Service ClusterIPs (e.g. <code>10.96.0.10</code> for kube-dns) are handed out by the apiserver when you create a Service of any type. Becomes the apiserver's <code>--service-cluster-ip-range</code> .
<code>pods.cidrBlocks</code>	kube-controller-manager (node-ipam-controller) first, then the CNI on each node	Two-tier IPAM: KCM splits the cluster-wide pod CIDR into per-node slices (typically <code>/24</code>) and writes each slice to <code>node.spec.podCIDR</code> . On each node, the CNI (Antrea by default in TKG) then allocates individual pod IPs from that node's slice when pods are scheduled there.

You can see the per-node slices after the cluster is up:

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.podCIDR}{"\n"}{end}'
# tkc-01-control-plane-...    <slice>/24
# tkc-01-node-pool-1-...      <slice>/24
# tkc-01-node-pool-1-...      <slice>/24
```

So:

- **Service IPs** are apiserver-allocated, IPAM is built into the control plane (no separate component).
- **Pod IPs** flow through two allocators: KCM at cluster-to-node granularity, CNI at node-to-pod.

CIDR sizing gotchas

⚠ **Avoid RFC 5737 ranges for the pod CIDR.** Address space `192.0.2.0/24`, `198.51.100.0/24`, and `203.0.113.0/24` is reserved for documentation and may collide with examples in other tools. The example uses `10.244.0.0/16` (Flannel/Calico convention). Other reasonable choices:

- `10.244.0.0/16` (Calico / Flannel default)
- `10.0.0.0/16`
- Any private range from RFC 1918 (`10/8`, `172.16/12`, `192.168/16`) that doesn't overlap your physical networks

A reasonable production-ish set:

```
clusterNetwork:
  services:
    cidrBlocks: ["10.96.0.0/16"]      # 65k cluster IPs — apiserver
  pods:
    cidrBlocks: ["10.244.0.0/16"]    # 65k pod IPs total
  serviceDomain: cluster.local
```

The cluster pod CIDR sets a hard ceiling on how many pods the cluster can run (minus per-node slice overhead). A `/16` gives ~250 nodes × 250 pods/node, more than enough for any lab.

Step 4 — watch the rollout

```
# Cluster + machine summary:
kubectl get cluster,machine -n test-ns
# Expect: Phase transitions Provisioning → Provisioned → Running
# Typical timing on lab hardware: 10-20 min

# Detailed status (find the stuck step if it stalls):
kubectl describe cluster tkc-01 -n test-ns
kubectl describe kubeadmcontrolplane -n test-ns
kubectl describe machinedeployment -n test-ns

# vSphere UI parallel views:
#   Workload Management → Namespaces → test-ns → Compute → Tanzu Kubernetes Clusters
#   Hosts and Clusters → Supervisor-Cluster → Namespaces → tkc-01 → VMs
```

Sequence you'll see:

1. **Provisioning** — Supervisor creates VMs for the control plane
2. **Provisioned** — VMs come up, kubeadm-init runs on the first CP
3. **Running** — workers join, networking + storage finishes wiring up

Step 5 — log into the workload cluster

5.1 — confirm the cluster's API server is reachable

Two checks before attempting login. The login itself will hang if either is missing.

```
# (a) KubeadmControlPlane shows API server is up.
# Run from the Supervisor namespace context (test-ns):
kubectl get kubeadmcontrolplane -n test-ns
# Look for: INITIALIZED=true AND API SERVER AVAILABLE=true

# (b) The workload cluster's API VIP.
# This is a SEPARATE VIP from the Supervisor's API endpoint.
# The Supervisor allocates one for each TKG cluster from its
# VIP pool (192.168.3.249-254 in this lab).
kubectl get cluster tkc-01 -n test-ns \
  -o jsonpath='{.spec.controlPlaneEndpoint}{"\n"}'
# Example: {"host":"192.168.3.252","port":6443}
```

A **fully ready** cluster looks like this (using the admin path from the “Alternate path” section below for the cleanest output):

```
=== Cluster ===
NAME          CLUSTERCLASS      PHASE      AGE      VERSION
tkc-01        builtin-generic-v3.1.0  Provisioned  11h      v1.32.0+vmware.6-fips

=== KubeadmControlPlane ===
NAME          CLUSTER  INITIALIZED  API SERVER AVAILABLE  REPLICAS  READY
UPDATED  UNAVAILABLE
tkc-01-gdhq8  tkc-01   true        true                  1          1
1           0

=== Machines ===
NAME          PROVIDERID          PHASE          CLUSTER  NODENAME
tkc-01-gdhq8-sk4j7  vsphere://...      Running        tkc-01   tkc-01-gdhq8-sk4j7
tkc-01-node-pool-1-fwf2k-lxs57-...  vsphere://...      Running        tkc-01   tkc-01-node-pool-1-fwf2k-lxs57-...
tkc-01-node-pool-1-fwf2k-lxs57-...  vsphere://...      Running        tkc-01   tkc-01-node-pool-1-fwf2k-lxs57-...
tkc-01-node-pool-1-fwf2k-lxs57-...  vsphere://...      Running        tkc-01   tkc-01-node-pool-1-fwf2k-lxs57-...
```

```
vsphere://...      Running

=== API endpoint ===
{"host": "192.168.3.252", "port": 6443}
```

Signals to verify:

- **Cluster** phase = `Provisioned` (note: `Provisioned`, not `Running` — `Running` shows up in some vSphere versions but in others the Cluster CR stops at `Provisioned` even when fully up)
- **KubeadmControlPlane**: `INITIALIZED=true` AND `API SERVER AVAILABLE=true`, `READY` ≥ 1
- **Machine** phases: all `Running`, with `NodeName` + `ProviderID` populated (= kubeadm-join completed successfully on each)
- No `Conditions` with `status=False` other than informational ones — use `kubectl describe cluster tkc-01 -n test-ns` to read the full condition list

5.2 — the kubectl-vsphere login command

```
kubectl vsphere login \  
--server=k8s.skynetsystems.io \  
--insecure-skip-tls-verify \  
--vsphere-username=administrator@vsphere.local \  
--tanzu-kubernetes-cluster-namespace=test-ns \  
--tanzu-kubernetes-cluster-name=tkc-01
```

What the flags do:

Flag	Why
<code>--server=</code>	The Supervisor's API endpoint (NOT the workload cluster's). The plugin authenticates against the Supervisor's wcp-login service and fetches a kubeconfig for the workload cluster from there.
<code>--insecure-skip-tls-verify</code>	The workload cluster's API cert has an IP SAN only, no hostname. Without this flag, TLS validation fails.
<code>--vsphere-username=</code>	vSphere SSO user. Must have the <code>Owner</code> (or <code>Edit</code>) role on the namespace from Step 1.
<code>--tanzu-kubernetes-cluster-namespace=</code>	The vSphere Namespace that owns the cluster. Matches <code>metadata.namespace</code> in <code>tkc.yaml</code> .

Flag	Why
<code>--tanzu-kubernetes-cluster-name=</code>	The cluster name. Matches <code>metadata.name</code> in <code>tkc.yaml</code> .

Result: a new kubeconfig context named after the cluster (e.g. `tkc-01`) is added to `~/.kube/config`. Your existing Supervisor contexts stay.

5.3 — switch context and verify

```
kubectl config use-context tkc-01

kubectl get nodes
# Expect (matches your tkc.yaml):
#   NAME                                STATUS    ROLES    AGE   VERSION
#   tkc-01--<n>                      Ready     control-plane  ...
#   tkc-01-node-pool-1--<id>-<n>    Ready     <none>    ...
#   tkc-01-node-pool-1--<id>-<n>    Ready     <none>    ...
#   tkc-01-node-pool-1--<id>-<n>    Ready     <none>    ...
#   tkc-01-node-pool-1--<id>-<n>    Ready     <none>    ...

kubectl get pods -A      # confirm system components are running
```

You're now in a real Kubernetes cluster — deploy whatever you'd normally deploy.

Caveats

External access — auth path vs data plane path

`kubectl vsphere login` and subsequent `kubectl get ...` use **two different endpoints**:

Path	Endpoint	Used for
Auth	Supervisor's API VIP, e.g. <code>192.168.3.250:443</code>	<code>kubectl vsphere login</code> itself — SSO, fetching the kubeconfig
Data plane	Workload cluster's API VIP, e.g. <code>192.168.3.252:6443</code>	Every <code>kubectl get/apply/exec/...</code> after login

A router port-forward that handles only the Supervisor (`:443` → `192.168.3.250`) gets you logged in from outside, but every subsequent `kubectl` call times out reaching `.252:6443`.

Three options for external use:

Option	What	Tradeoff
Login from inside the LAN (or VPN/Tailscale)	No router change	Requires being on-network
Add a second port forward <code>:6443</code> → <code>192.168.3.252:6443</code>	External <code>kubectl</code> works after login	One extra rule, plus exposes K8s API publicly — guard with strong auth + ideally an IP allowlist on the router
Login externally, accept that <code>kubectl</code> won't work afterward	Only does the auth step; you'd manually switch to a context after VPN-ing in	Confusing in practice; usually you want one or two above

The Supervisor port-forward handles the **authentication** path. A separate `.252:6443` forward handles the **data plane** path for actual cluster operations.

Other gotchas

- **API server availability lags** `INITIALIZED=true` **by 30-60 sec.** If `INITIALIZED=true` but `API_SERVER_AVAILABLE=false`, give it a minute before retrying login.
- `--insecure-skip-tls-verify` **is required** until you provide a proper cert with a hostname SAN. The default cert has only the VIP as a SAN.
- **Each TKG cluster gets its own VIP** from the Supervisor's VIP pool. If you spin up `tkc-02` later, it'll land on a different IP (probably `.253` or whichever's next free). Re-run the `kubectl get cluster <name> -n <ns> -o jsonpath='{.spec.controlPlaneEndpoint}'` query to find each cluster's endpoint.

Alternate path: admin access via a Supervisor CP VM

When the SSO login path isn't working (e.g. DNS issues, namespace RBAC in flux during early cluster bring-up), you can also reach the Supervisor's K8s API directly by SSH'ing into a **Supervisor Control Plane VM**, which has cluster-admin access via a local kubeconfig:

```
# 1. Get the Supervisor CP VM root password (rotates every ~24h):
cd /Users/ben/Repos/greylog/terraform
make cp-pwd
# Prints IP: 192.168.2.231 (floating) + PWD: <password>

# 2. SSH into any one of the CP VMs (.231 floating, .232/.233/.234 the
#   three CP nodes). Pick a stable IP rather than the floater so the
#   connection survives a CP rotation:
sshpass -p '<password>' ssh root@192.168.2.232

# 3. On the CP VM, point KUBECONFIG at admin.conf and query the
#   Supervisor's K8s API (which holds the Cluster + Machine CRs):
```

```
export KUBECONFIG=/etc/kubernetes/admin.conf
kubectl get cluster -n test-ns
kubectl get machine,virtualmachine -n test-ns
kubectl describe cluster tkc-01 -n test-ns
```

This is the *admin* path — bypasses SSO entirely. Useful for:

- Diagnosing why `kubectl vsphere login` itself fails (this is how we debugged the DNS hijack earlier in the lab buildout)
- Seeing CAPI / CAPV / VM Operator controller logs:

```
kubectl get pods -A | grep -E 'capi|capv|capw|vmop'
kubectl logs -n <namespace> <pod>
```

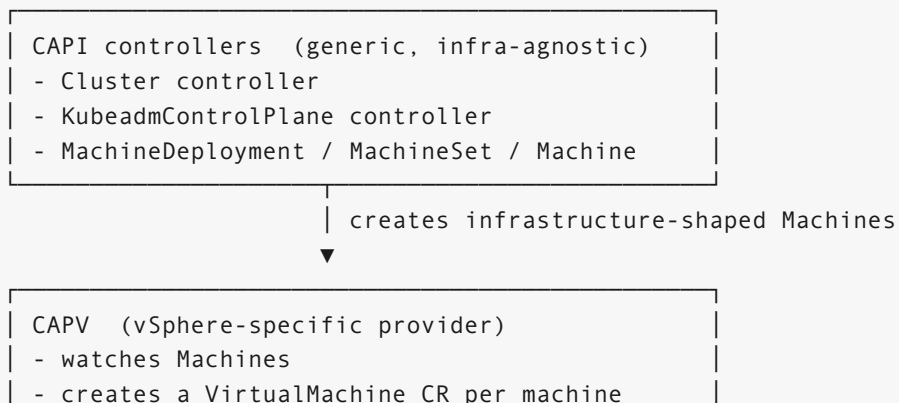
- Reading raw CR `status.conditions` fields that user RBAC may strip
- Querying CRDs that aren't surfaced via the namespace-scoped user view (e.g. `tanzukubernetesreleases`, cluster-wide controllers)

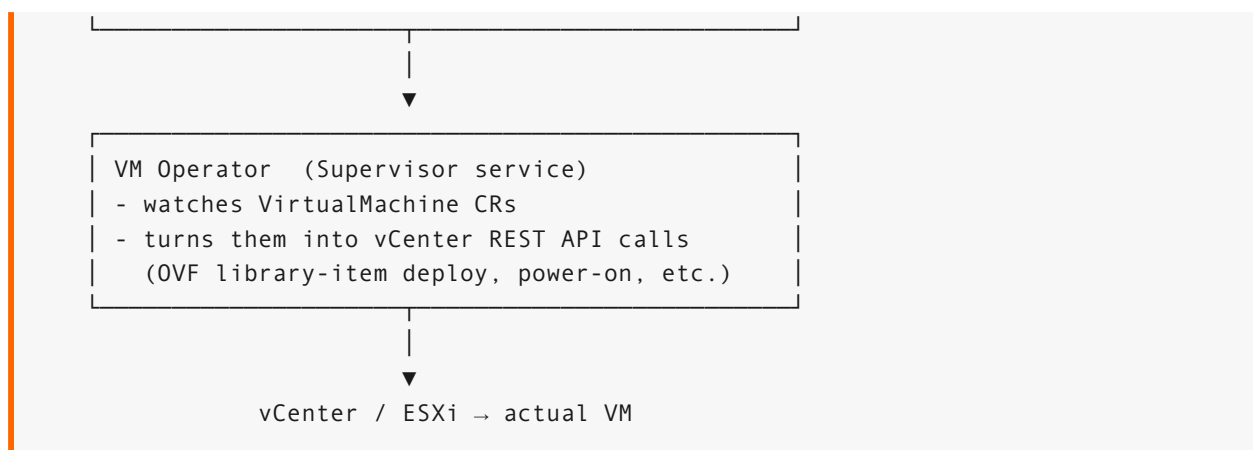
Important: the CP VM's `admin.conf` is for the **Supervisor**, not the workload cluster. To run workloads *on* the TKG cluster you need the SSO-issued kubeconfig from Step 5.2 — the `admin.conf` only sees Supervisor-level resources (CRs like `Cluster`, `Machine`, vSphere Namespaces, etc.).

How it actually works under the hood

When you `kubectl apply -f tkc.yaml`, you're not directly creating VMs. You're handing a high-level intent (`Cluster`) to a chain of controllers that translate it down to vSphere API calls. The chain:

You apply: Cluster CR (your `tkc.yaml`)





Glossary

Component	Full name	Role
CAPI	Cluster API	Upstream Kubernetes project for declaratively managing K8s clusters. Infrastructure-agnostic — defines <code>Cluster</code> , <code>Machine</code> , <code>KubeadmControlPlane</code> , <code>MachineDeployment</code> etc. as generic CRs.
CAPV	Cluster API Provider for vSphere	The vSphere-specific implementation of CAPI. Translates generic <code>Machine</code> resources into vSphere-shaped requests.
CAPW	Cluster API for Workload (legacy <code>wcp</code> naming)	Older name for the same role, still used in some controller namespaces (e.g. <code>vmware-system-capw-controller-manager</code>).
VM Operator	(vSphere Supervisor Service)	The component inside the Supervisor that actually calls vCenter. Owns the <code>VirtualMachine</code> , <code>VirtualMachineClass</code> , <code>VirtualMachineImage</code> CRDs.

Why this layering matters when debugging

When a TKG cluster gets stuck, the failure is usually at a *specific layer*. Knowing the chain tells you which controller's logs to look at:

Symptom	Which layer	Where to look
<code>Cluster</code> stays in <code>Provisioning</code> indefinitely, no <code>Machine</code> s created	CAPI Cluster controller can't satisfy the topology	<code>kubectl logs -n <cap[iv]-ns> deployment/capi-controller-manager</code>
<code>Machine</code> s created but no <code>VirtualMachine</code> CR appears	CAPV not responding	<code>kubectl logs -n vmware-system-capw deployment/capv-controller-manager</code> (namespace varies by version)
<code>VirtualMachine</code> CR shows <code>VirtualMachineCreated: False</code> with REST error	VM Operator → vCenter	<code>kubectl logs -n vmware-system-vmop deployment/vmoperator-controller-manager</code> + vCenter's <code>/var/log/vmware/content-library/cls.log</code> for OVF deploy errors
VM exists but stays powered off forever	VM Operator power-on stage	Same as above — usually a transient retry, or a hung OVF import session

The controller namespace names *change* across vSphere versions (`vmware-system-capw`, `vmware-system-capi`, `svc-tmc-...`, etc.). To find them on a live cluster:

```
# From a Supervisor CP VM with admin.conf:
kubectl get ns | grep -E 'cap[iv]|tmc|wcp|vmop'
kubectl get deploy -A | grep -E 'cap[iv]|vmoperator|tkg'
```

Storage policy → datastore landing

A side-effect of the chain above worth knowing: the **vSphere Namespace's storage policy** is what decides which datastore your TKG VMs land on. For this lab, the only datastore tagged for the `supervisor-storage` policy is `nfs-shared`, so:

- The default Kubernetes content library's backing storage is on `nfs-shared`
- All TKG control-plane + worker VM disks are created on `nfs-shared`

- The OVA “copy” you see in `cls.log` is server-side on the same NFS export — both source (content library cache) and target (new VM vmdk) live on the same datastore, so it’s a fast NFS-internal copy

If you tag a second datastore (e.g. vSAN) with `supervisor-storage`, vSphere will pick either when creating new VMs.

Troubleshooting

`kubectl get tkr` **returns nothing**

The namespace isn’t bound to the content library. In vSphere UI:

Namespaces → `test-ns` → *Summary* → *Tanzu Kubernetes Grid Service* → *Add Content Library* → pick `kubernetes.vmware.com`

Wait ~30s; `kubectl get tkr` should populate.

Cluster stuck in `Provisioning`

```
kubectl describe cluster <name> -n <ns> | grep -A20 'Conditions:'  
kubectl describe kubeadmcontrolplane -n <ns>
```

Common causes:

Symptom	Cause	Fix
“No matching VirtualMachineImage”	<code>version:</code> doesn’t match an available <code>tkr</code>	Pin to an exact value from <code>ku bectl get tkr</code>
“Failed to validate storage policy”	<code>storageClass:</code> not visible to the supervisor cluster	Confirm <code>supervisor- storage</code> tag still attached to <code>nfs-shared</code> (<code>govc tags.attached.ls supervisor-storage</code>)
“Failed to pull image from content library”	Library out of sync, or namespace not bound	Re-sync library (vSphere UI), then <code>kubectl delete</code> the cluster and re-apply
	CNI / IPAM still wiring up	Wait 2-3 min after <code>Running</code> ; if stays empty, <code>kubectl</code>

Symptom	Cause	Fix
Cluster <code>True</code> but <code>kubectl get nodes</code> empty / NotReady		<code>describe node</code> for kubelet issues

`kubectl vsphere login` **for the TKG cluster fails**

The `--tanzu-kubernetes-cluster-*` flags require the Supervisor login flow to succeed first (you should already have a `test-ns` context working). Make sure `test-ns` shows up in `kubectl config get-contexts` *before* trying the TKG login form.

Cleanup — delete the cluster

```
kubectl delete cluster tkc-01 -n test-ns
# Watch: kubectl get cluster -n test-ns -w
# Expect: Deleting → (gone) over ~5-10 min
```

This tears down all CP + worker VMs and frees the storage. Don't `kubectl delete` the underlying `machine` / `kubeadmcontrolplane` objects directly — let the top-level Cluster controller handle the cascade.

Pinning specific TKG version: how to read `kubectl get tkr`

NAME COMPATIBLE	VERSION	READY	
v1.28.8---vmware.1-fips.1-tkg.1	v1.28.8+vmware.1-fips.1-tkg.1	True	True
v1.27.13---vmware.1-fips.1-tkg.1	v1.27.13+vmware.1-fips.1-tkg.1	True	True
v1.26.13---vmware.1-fips.1-tkg.1	v1.26.13+vmware.1-fips.1-tkg.1	True	True

- **NAME** uses `---` as the version separator (not `+` — DNS-safe)
- **VERSION** uses `+` and is what you put in `topology.version:`
- **READY=True + COMPATIBLE=True** are both required; skip any with `False` in either column
- Older clusters can be upgraded by editing `topology.version` to a newer tkr; the Cluster controller rolls workers one at a time

Future automation

Today these steps are manual. A `terraform/modules/workload-cluster` that takes (namespace, name, version, sizing) as inputs and uses the Kubernetes provider's `kubectl_man`

`fest` would let you spin up named clusters with `terraform apply`. The `terraform/modules/content-library` module already exists as a stub for managing the subscribed library, so the pieces are most of the way there.

Until that lands, copy this doc's `tkc.yaml`, fill in the version + a unique `metadata.name`, and apply.